pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Batch Size, Epochs, and Loss Curves

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Three settings, two curves

Batch size

How many rows each step looks at

Epochs

How many passes over the fit rows

Early stopping

When to end the fitting, by a callback

Two curves, recorded every epoch

Training loss: the 14,400 fit rows

Stopping loss: the 3,600 stopping rows

Every setting judged on the stopping rows

Validation rows (6,000): report only

Last section: the seed spread, the yardstick for every comparison after today.

How batches, steps, and epochs work

1. Shuffle

the fit rows, each epoch

2. Cut into batches

batch_size rows each

3. One step per batch

the gradient from that batch's rows

4. An epoch

one pass through every batch

You control

The batch size and the number of epochs; the steps follow.

What comes out

Weights after every step; a loss after every epoch.

An epoch, a batch, a step

Batch size	Steps per epoch
32	450
64	225
256	57
1,024	15
14,400	1

Three words

A batch: the rows one step's gradient is worked out from. A step: one move of every weight. An epoch: one pass through all the batches.

```
netkit.steps_per_epoch(14_400, 64)   # ceil
```

At the same epochs, a smaller batch takes more steps, each from fewer rows.

How loss curves work

1. Each epoch

Keras records two numbers

2. loss

on the rows it fits

3. val_loss

on the stopping rows, never fitted

4. Against the epoch

the two loss curves

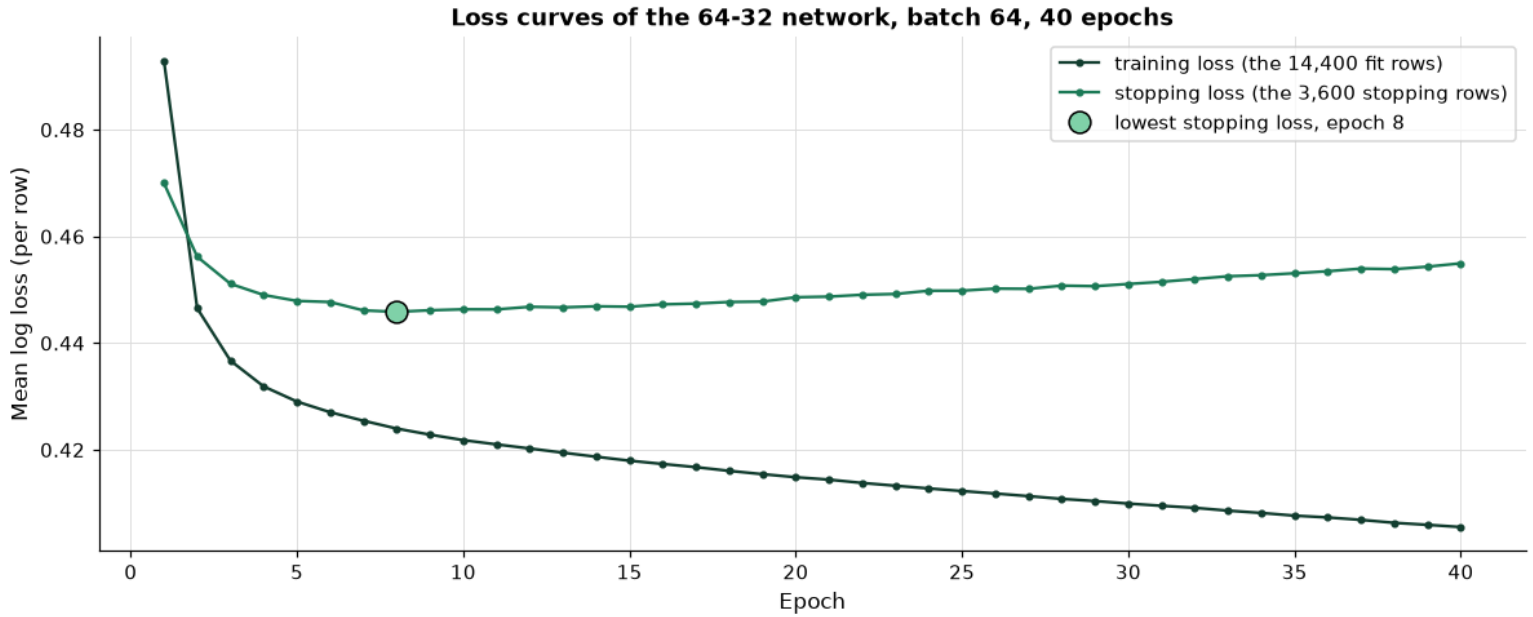
You control

Which rows are passed as the stopping rows.

What comes out

The History: one number per epoch for each curve.

Two loss curves, one network



Training loss falls every epoch (0.4930 to 0.4055); stopping loss: 0.4700, lowest 0.4459, then 0.4550.

How the batch size works

1. Small batch

noisy gradient, many steps per epoch

2. Large batch

steadier gradient, few steps

3. Fixed epochs

a small batch takes far more steps

4. Fair test

each at its own lowest stopping loss

You control

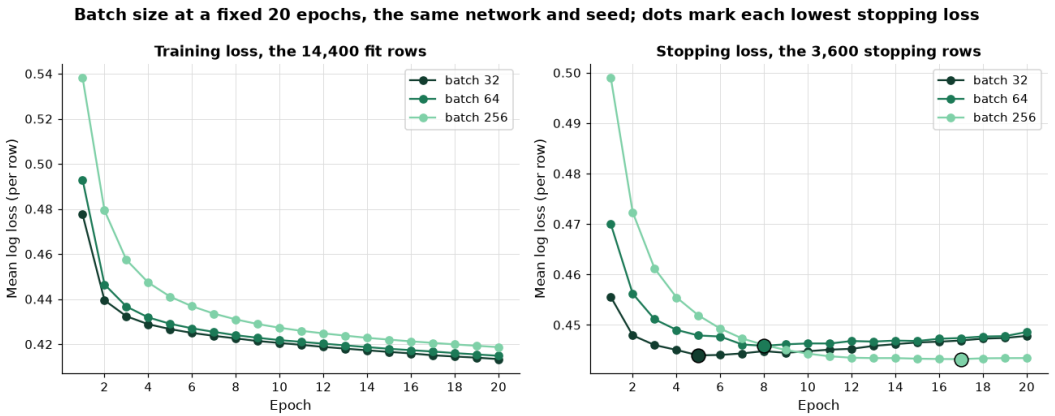
The batch size; everything else stays the same.

What comes out

A loss curve per batch size.

Batch size at a fixed 20 epochs

Batch	Steps in all	Train loss, ep. 20	Stop loss, ep. 20	Lowest stop loss	At epoch
32	9,000	0.4134	0.4478	0.4440	5
64	4,500	0.4149	0.4486	0.4459	8
256	1,140	0.4187	0.4434	0.4432	17



Compare each batch at its own best epoch: smaller batches fit more per epoch and turn sooner.

The one cell outside the size

```
# thread tolerance: large batches differ in the last digits across thread counts
batch 1,024: 300 steps in all; training loss 0.430, stopping loss 0.446
batch 14,400: 20 steps in all; training loss 0.534, stopping loss 0.532
```

Why the last digits move

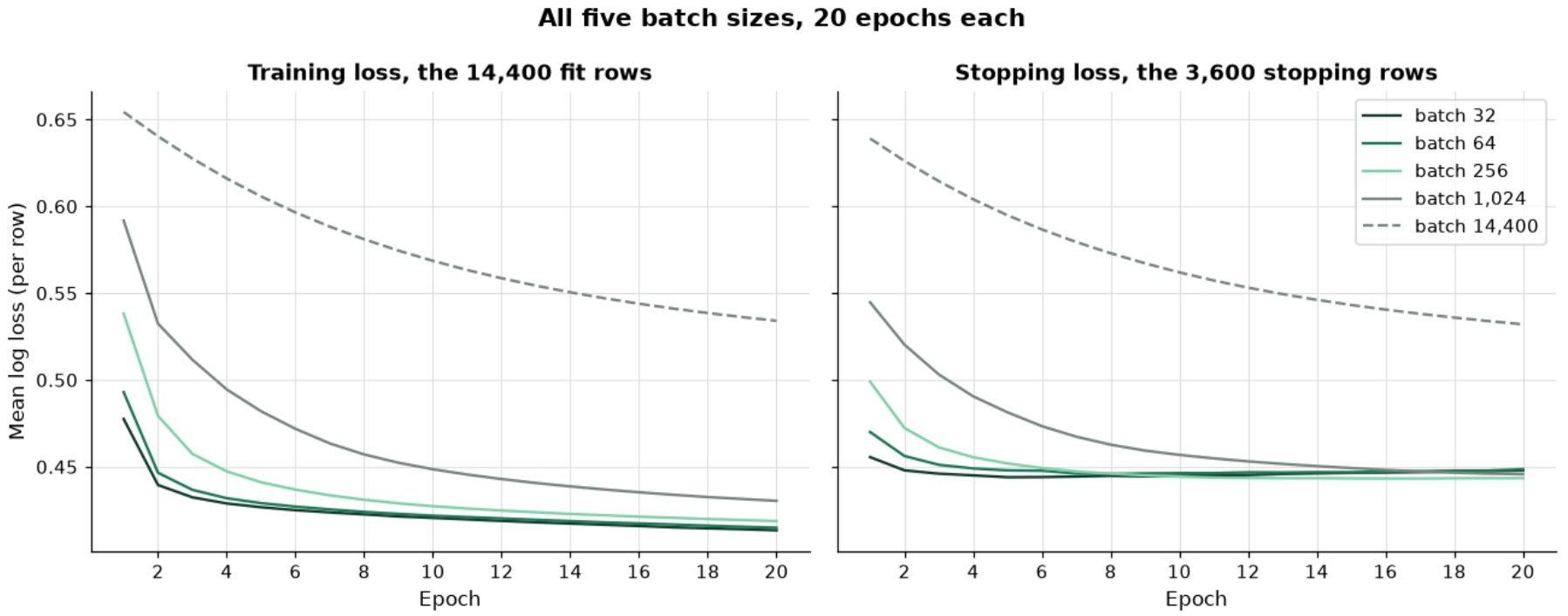
A large batch is a large product. On every core TensorFlow adds its parts in a different order, and float32 sums differ in the last digits.

The rule for this one cell

Marked on its first line, printed to 3 decimals, allowed to move by up to 0.005. Every other cell matches exactly; nothing later uses it.

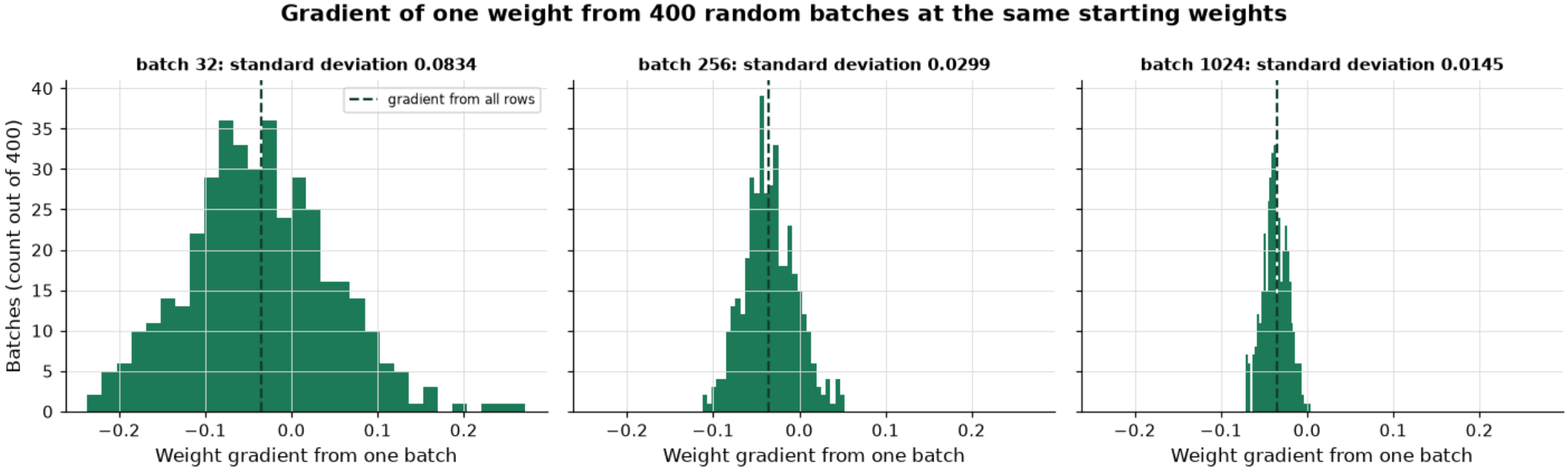
Larger batches take fewer steps per epoch, so at the same epochs they move the weights less.

All five batch sizes, 20 epochs each



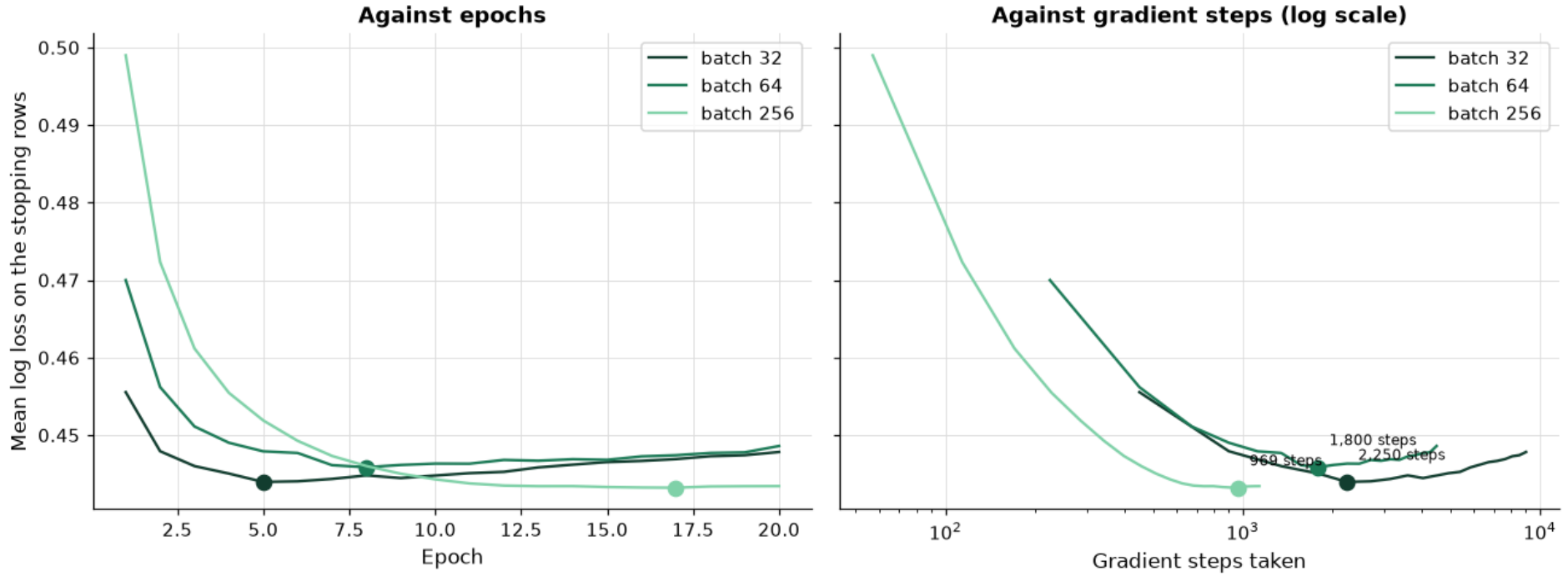
Batch 1,024 (300 steps) and the full batch (20 steps) are still falling at epoch 20: count the steps, not the epochs.

A batch's gradient is noisy



Standard deviation: batch 32 0.0834, batch 256 0.0299, batch 1,024 0.0145. A batch eight times larger cuts the scatter by about the square root of 8.

Loss against steps, not epochs



Lowest point after 2,250 steps at batch 32, 1,800 at 64, 969 at 256: a larger batch needs fewer steps. Steps count updates, not time.

How the best epoch is found

1. Read val_loss

one number per epoch

2. Take the smallest

counting epochs from 1

3. Ties

the first epoch wins

4. Not training loss

it keeps falling

You control

The number of epochs allowed.

What comes out

One epoch number.

The best epoch is read on the stopping loss

8

best epoch on the stopping loss, of 40

40

the "best" epoch on the training loss: the last

	Epoch 8	Epoch 40
Stopping loss	0.4459	0.4550
Training loss	0.4240	0.4055

netkit.best_epoch

np.argmin of val_loss, plus 1: the first epoch with the smallest stopping loss.

The training loss keeps falling for as long as you fit: it cannot pick the epoch.

How early stopping works

1. Watch val_loss

after every epoch

2. Remember

the epoch with the best value so far

3. Count

epochs since it last improved

4. Stop and restore

at patience; put back the best weights

You control

The monitored value, the patience, and the maximum epochs.

What comes out

A model holding the best epoch's weights; a shorter History.

Early stopping: the epoch it picked, the weights it kept

```
keras.callbacks.EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)
model.fit(..., epochs=200, batch_size=64, callbacks=[stopper], verbose=0)
```

18

epochs run, of up to 200

8

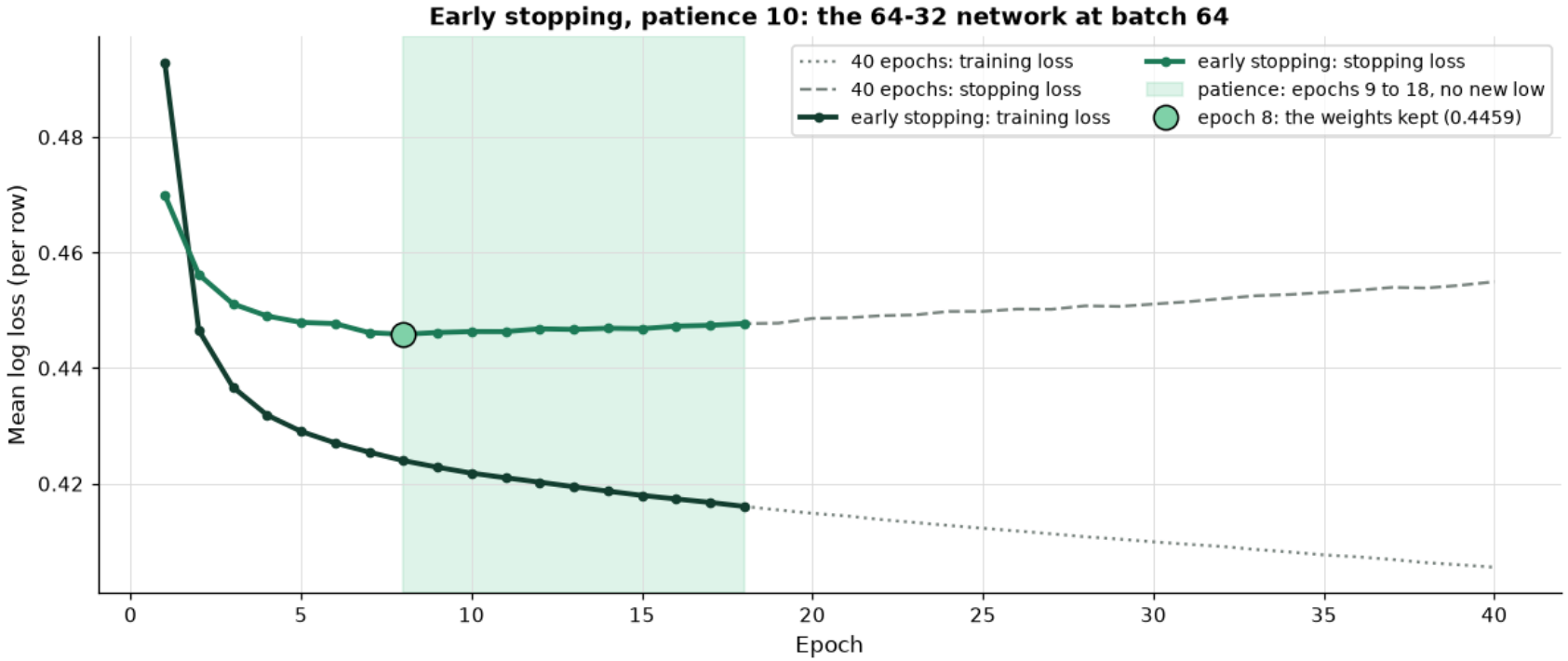
epoch kept (EarlyStopping's best_epoch + 1: 8)

0.4459

stopping loss of the weights kept (smallest: 0.4459)

The same run as section 4.2, epoch for epoch, until it stopped: early stopping changed when the fitting ended.

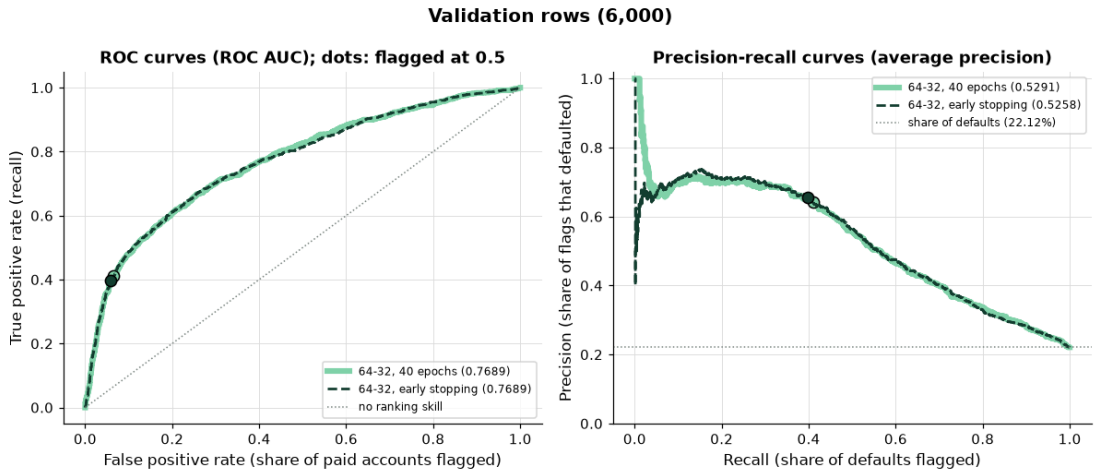
Early stopping on the curve



The same curve, cut short at epoch 18; the weights of epoch 8 put back (0.4459).

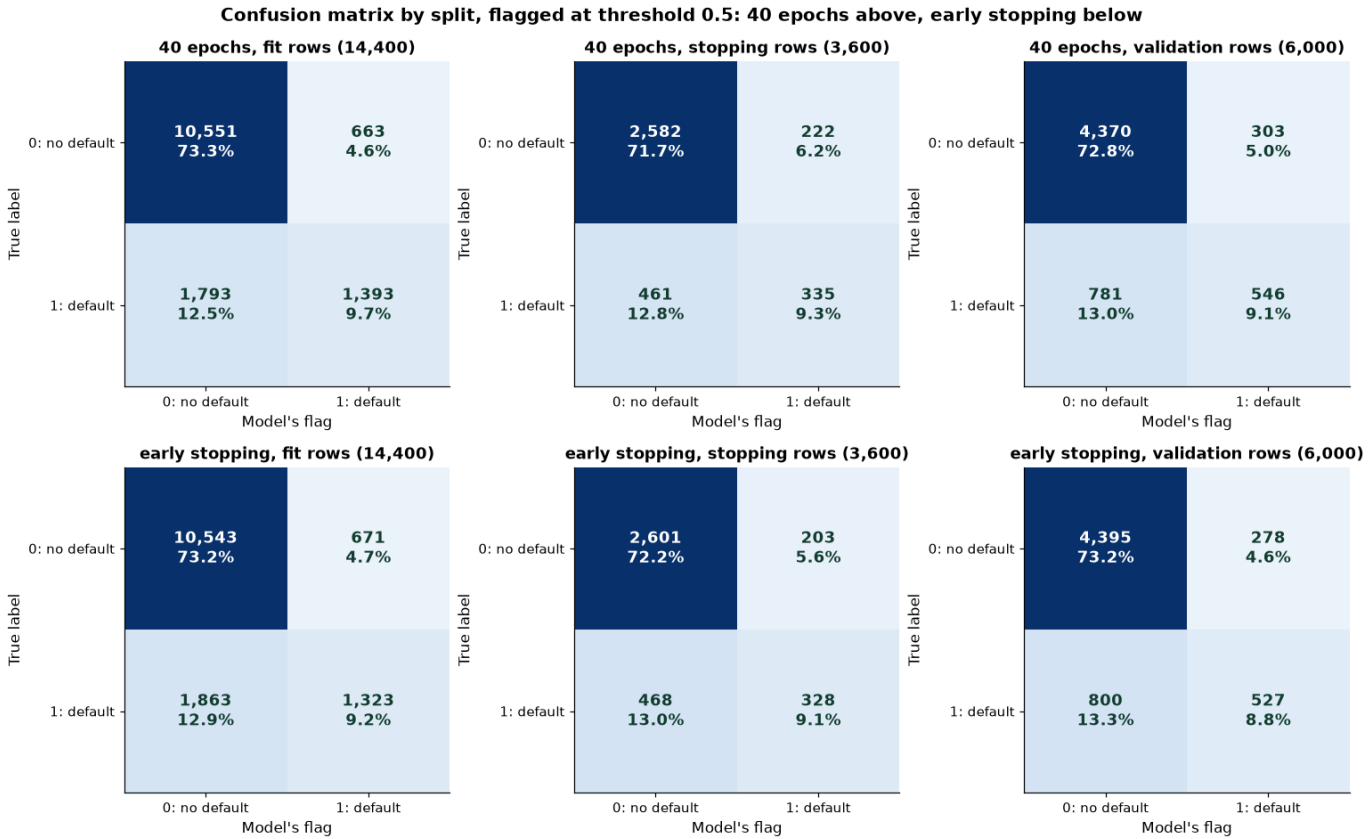
Validation: the same ranking, a better log loss

Validation rows	ROC AUC	Average precision	Mean log loss
64-32, 40 epochs	0.7689	0.5291	0.4460
64-32, early stopping	0.7689	0.5258	0.4430



Reported once each. The epoch was chosen on the stopping rows, never on these.

Confusion matrices by split, at 0.5



849

validation accounts flagged, 40 epochs

805

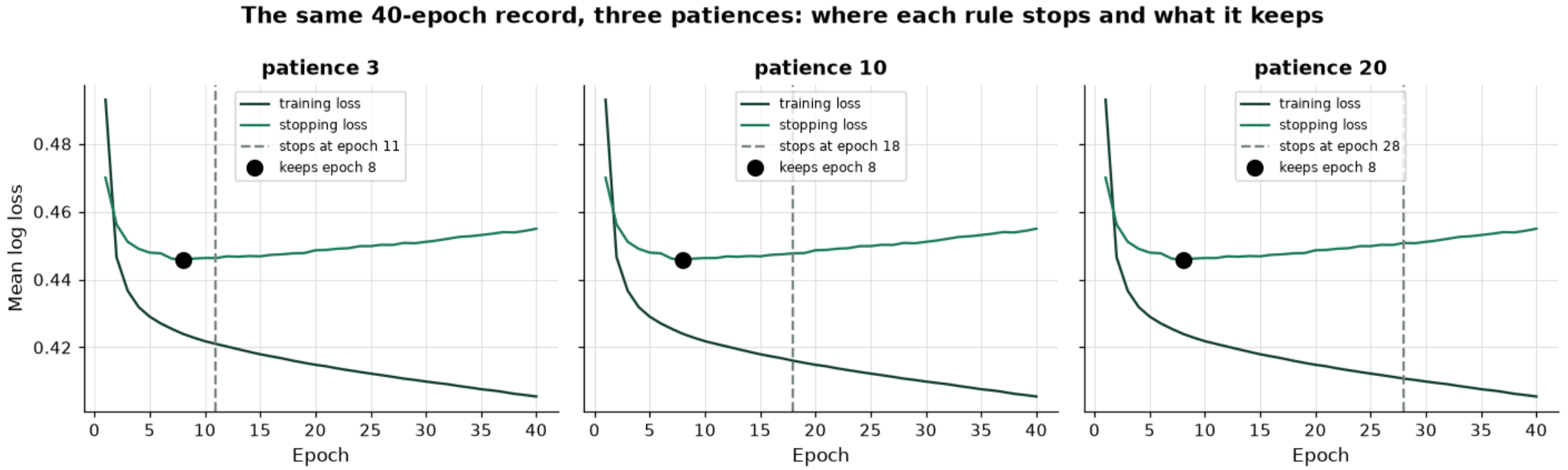
flagged, early stopping

Recall, fit rows against validation

40 epochs: 43.7% against 41.1%. Early stopping: 41.5% against 39.7%.

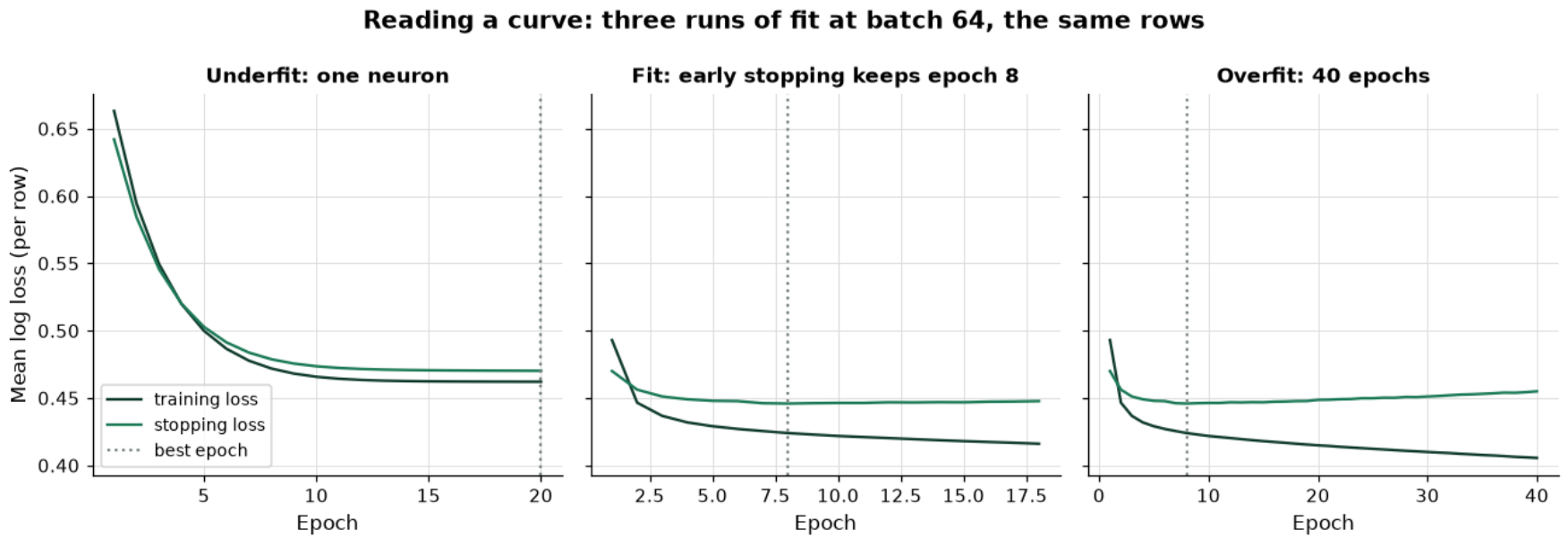
Fitting past the best epoch shows first on the fit rows.

How much patience



Patience 3 stops at epoch 11, 10 at 18, 20 at 28; all keep epoch 8. Pick the smallest patience that does not stop on a dip the curve would leave.

Reading a curve: underfit, fit, overfit



The neuron ends at 0.4621 and 0.4702, still falling; its lowest stopping loss 0.4702 is above the network's 0.4459.

Where a GPU would help

Large products

A step multiplies each batch by each layer's weights. A GPU does many multiplications at once, so it helps with large batches through wide layers.

This course's networks

On the card file: 19 inputs, hidden layers of 64 units or fewer, batches of 256 or fewer outside one marked cell. Small products: the CPU is enough.

Colab: Runtime, Change Runtime Type. Never required; nothing here was run or timed on a GPU.

Day 1's devices paragraph is repeated word for word in the notebook, with its sources (TensorFlow 2.21.0's config.py; Colab's FAQ, read 6 October 2026).

How the seed spread works

1. A seed

fixes the starting weights and the batch order

2. Another seed

another network, same recipe

3. Refit

the same network under several seeds

4. The spread

range and standard deviation of a score

You control

The seeds.

What comes out

The yardstick a difference between two settings has to beat.

Seeds: the yardstick for every comparison

Seed	Best epoch	Lowest stop loss	Validation ROC AUC
SEED + 0	8	0.4459	0.7689
SEED + 1	13	0.4471	0.7697
SEED + 2	12	0.4440	0.7701
SEED + 3	11	0.4486	0.7696
SEED + 4	8	0.4462	0.7712

0.0046

range of the lowest stopping loss (sd 0.0015)

0.0023

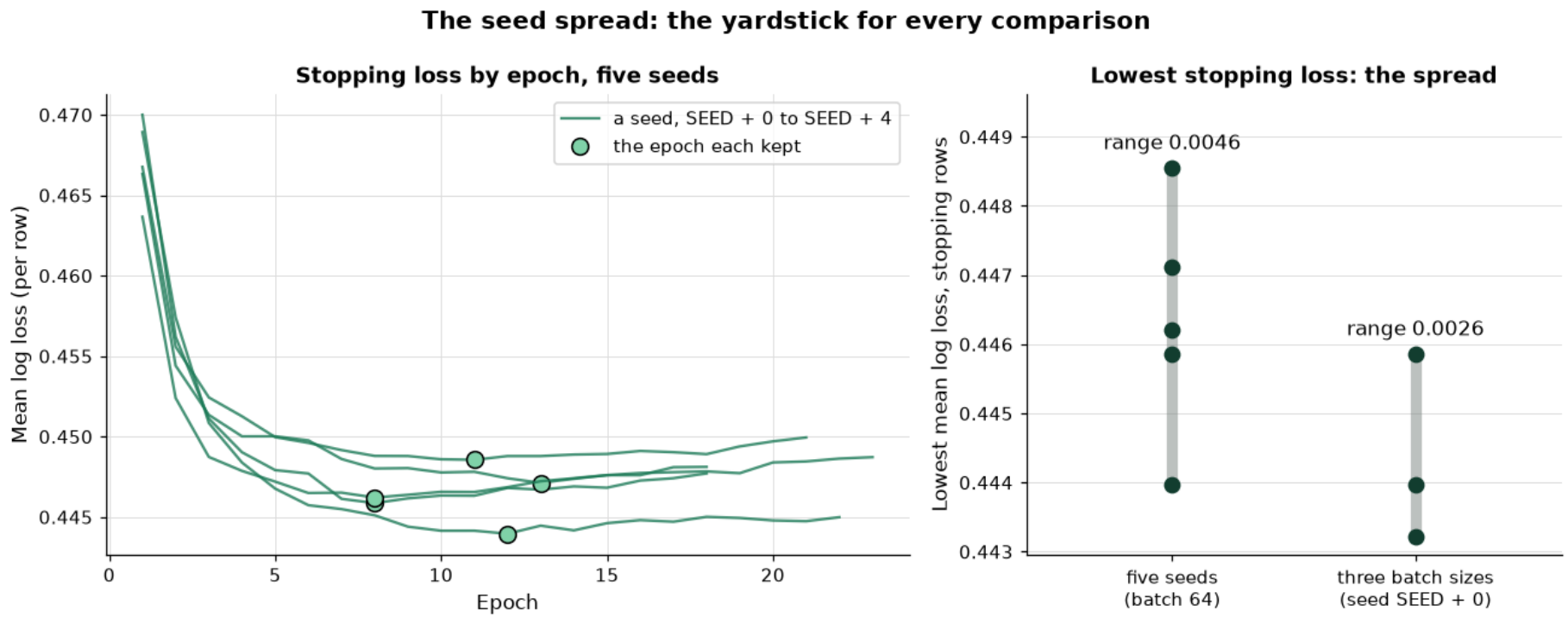
range of validation ROC AUC (sd 0.0007)

Section 4.3's batches

Their lowest stopping losses lie closer together than the seeds': a spread the seed alone can make.

Five validation scores measure the spread; picking a seed on them would be choosing on the report rows.

The seed spread, drawn



The seeds' range, 0.0046, is longer than the batch sizes': read every comparison against it.

Excel side by side

Excel side by side (tested in Excel)

`=ROUNDUP(14400/64,0)`	225 steps per epoch
`=ROUNDUP(14400/F2,0)`	filled down: 450, 225, 113, 57, 15, 1
`=MATCH(MIN(C2:C41),C2:C41,0)`	epoch 8, the best epoch
`=MIN(C2:C41)`	0.445864, the lowest stopping loss
`=MATCH(MIN(B2:B41),B2:B41,0)`	epoch 40: the AI check's mistake
`=C2-B2`	filled down: the gap, exercise 3's curve_gap
`=MATCH(MIN(K2:K5),K2:K5,0)`	2 on 0.7, 0.5, 0.6, 0.5: the first of a tie

No network training in Excel: it cannot make a loss curve, but it reads one. Equal to Python within 1e-12.

Save the day: git show HEAD~1

1. Modules

mlkit, ensemblekit, netkit and its tests

2. 40 epochs

results.csv, the first row

3. Early stopping

results.csv, a second row

```
!git -C "{work}" show HEAD~1:results.csv
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
dense_64_32_40_epochs,validation,0.7689,0.5291,0.5000,0.8193,0.6431,0.4115,0.5018
```

HEAD~1:results.csv is the file one commit back, folder unchanged. All-zero baseline accuracy 0.7788.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the epoch is read from the training loss: it returns 18, the last epoch run, where EarlyStopping kept 8.

Code that runs is not the same as code that is right.

What you can do now

- Count the steps in an epoch at any batch size
- Draw and read the training and stopping loss curves
- Judge each batch size at its own best epoch
- Find the best epoch on the stopping loss; stop early
- Read a comparison against the seed spread

Week 1, Day 5 Case study: a first network on credit card default